Kiran Khatri ID: 923750019
Github: khatri5034 CSC615 Embedded Linux

Assignment 3 – Motors and Motor Driver Hat

Description:
This assignment is about controlling a DC motor using the WaveShare Motor Driver HAT connected to the Raspberry Pi through I2C communication. The motor is controlled using Pulse Width Modulation (PWM) for both speed and direction.

When the program runs, it waits until the push button is pressed. Once the button is pressed, the motor runs forward at 100% speed for at least 2 seconds. Then the motor gradually slows down to 15%. After that, the motor stops for at least one second. Finally, the motor starts again in reverse direction and gradually increases speed until it reaches maximum speed.

The motor is powered using a separate 12v battery connected to the motor power input on the HAT.

Approach / What was my:

First, I will connect the Motor Driver HAT to the Raspberry Pi and verify that the I2C address appears using the i2cdetect command. I expect to see address 0x42 on the I2C bus.Next, I will connect a single DC motor to the MB terminals on the Motor Driver HAT. I will connect the 12 v battery to the motor power input on the HAT and make sure the polarity is correct.

Then I will connect a push button to GPIO17 (Physical Pin 11) and GND (Physical Pin 9). The program will configure this pin as input and wait until the button is pressed before starting the motor sequence.

Then, i will extract only the files i need for this assignment not SPI from the provided code and remove the codes/files that implement SPI and other protocols. I will initialize I2C communication using DEV_ModuleInit() and configure the PCA9685 chip using PCA9685_Init() and PCA9685_SetPWMFreq(). I will write my own motor control functions instead of using the provided MotorDriver.c file to control the motor as in the requirements using the pigpio library to control the switch.

Issues and Resolutions:

One major issue I encountered was related to power wiring. I accidentally connected the positive terminal from the wrong power source directly to the motor positive terminal on the HAT. This caused a short circuit and damaged my SD card.

I resolved it by consulting a professor who then provided me with a new SD card. After replacing the SD card, I carefully rechecked the wiring and made sure that the motor power supply was connected only to the motor power input with correct polarity.

Another issue occurred with the button input. Initially, the button readings were unstable and the program would start even when the button was not pressed. After debugging, I realized the GPIO input configuration was not correct and the pin was not stable.

I resolved it by correcting the GPIO setup and verified the button wiring, the program properly waited for the button press before starting the motor sequence.

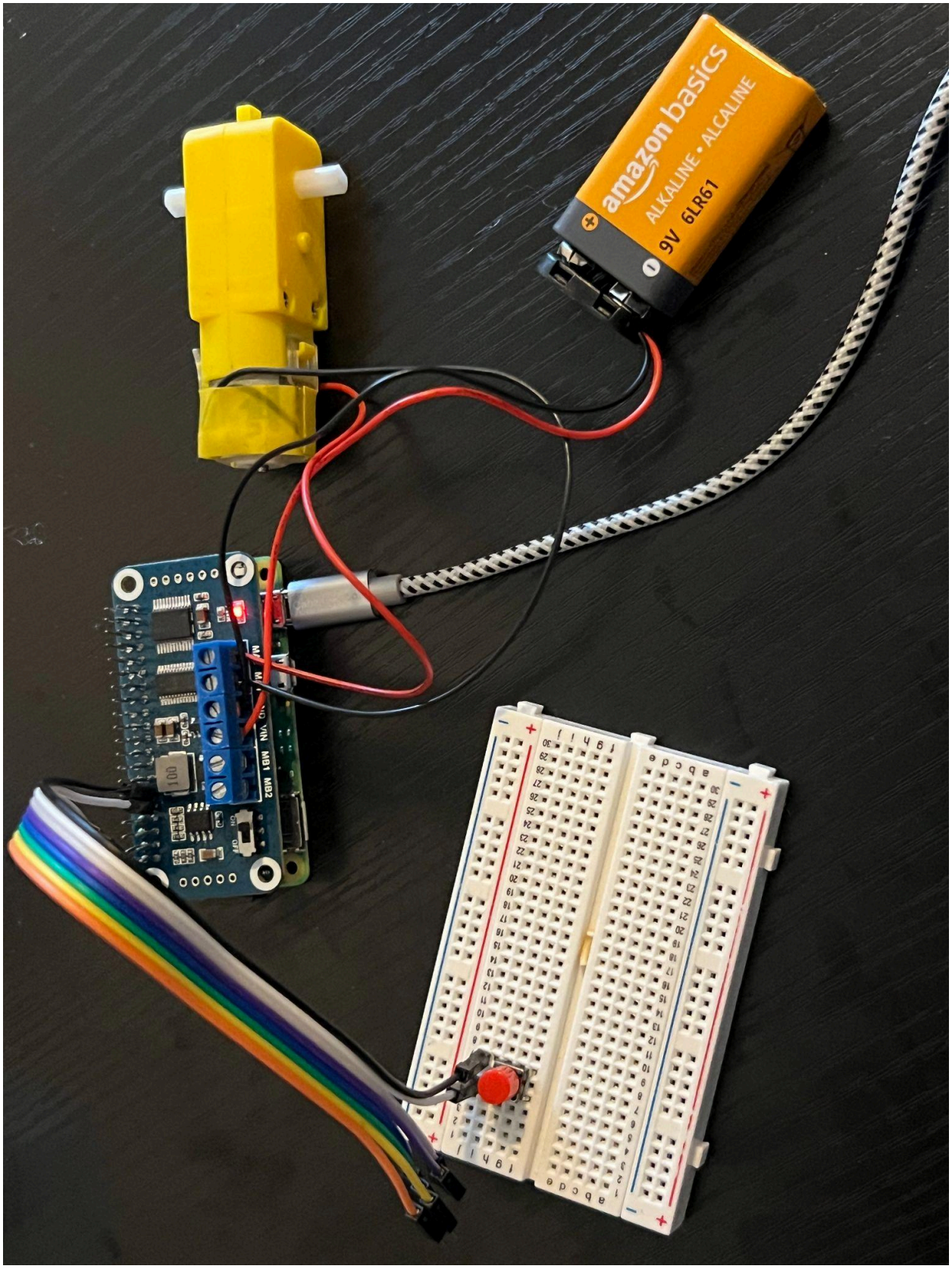
Another issue was I was passing the wrong value to control the duty cycle.

I resolved it by verifying passing the right value which is 100 to the PCA9685 driver, since the library already handles internal scaling. After confirming this, the speed control worked as expected.

Kiran Khatri
Github: khatri5034

ID: 923750019
CSC615 Embedded Linux

Photo of completed circuit:

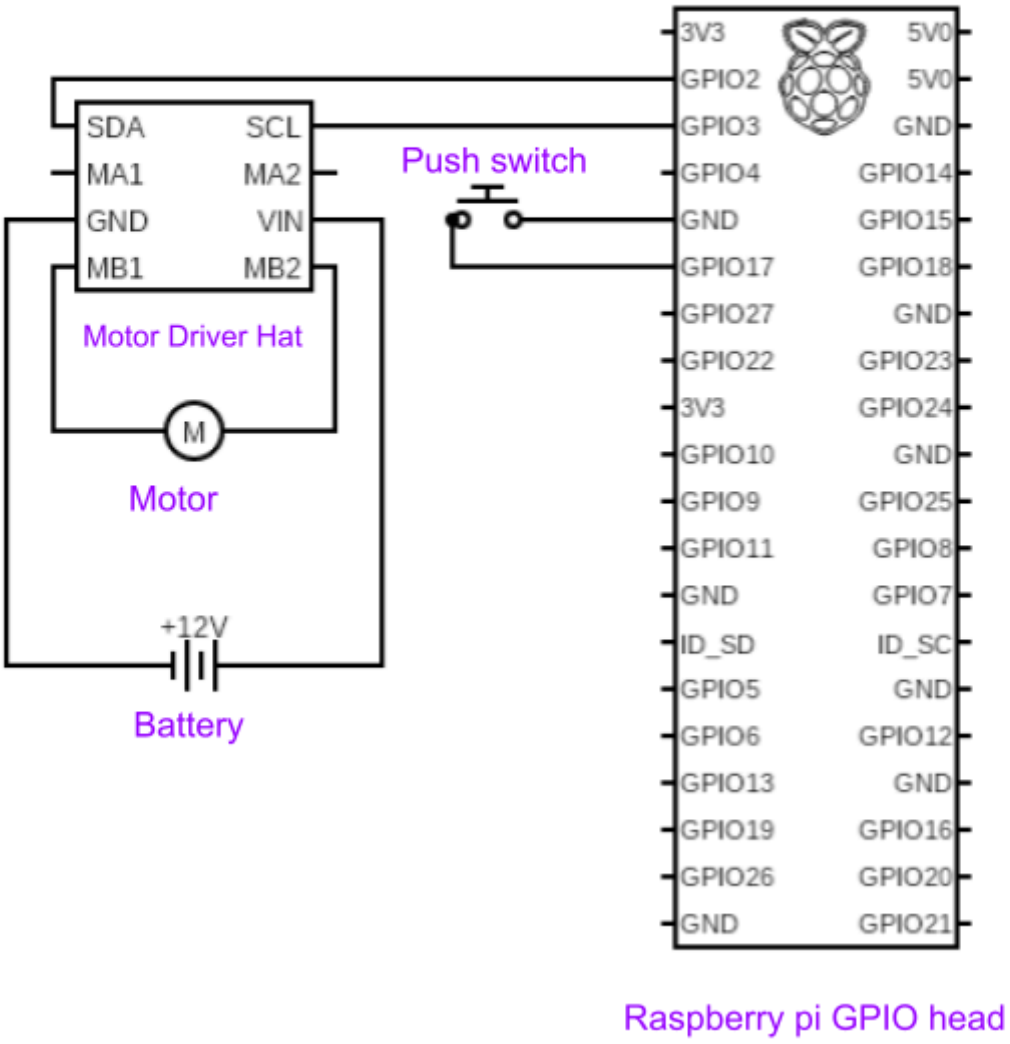


Kiran Khatri
Github: khatri5034

ID: 923750019
CSC615 Embedded Linux

Hardware Diagram:

SDA Line = GPIO 2 = Physical pin 3
SCL line = GPIO 3 = Physical pin 5
GND pin = Physical pin 9
GPIO 17 pin = Physical pin 11



Kiran Khatri
Github: khatri5034

ID: 923750019
CSC615 Embedded Linux

Analysis:

(If required for the assignment)

Generative AI:

Prompt: what would be the good PWM frequency for DC motor?

Response: For a typical small DC motor like the one you're using, a good PWM frequency is usually:

👉 100 Hz to 500 Hz

That range gives smooth control without stressing the driver.

Why 100 Hz works well:

- The motor runs smoothly.
- There is no visible pulsing.
- The PCA9685 handles it easily.
- It keeps good PWM resolution.

100 Hz is very common and completely acceptable.

What happens if it's too low (like 10–20 Hz)?

- The motor may jerk or vibrate.
- You can sometimes hear the switching clearly.
- It does not feel smooth.

What happens if it's too high (like 5 kHz or 10 kHz)?

- The PCA9685 may not calculate the prescaler correctly.
- Effective PWM resolution decreases.
- It gives no real benefit for a simple DC motor.
- It may increase switching losses.

Screen shot of compilation:

```
pi40@raspberrypi40:~/EmbeddingLinux/csc-615-assignment-3-startyourmotors-khatri5034 $ make
gcc -c -Wall -g -D USE_DEV_LIB  main.c -o main.o
gcc -c -Wall -g -D USE_DEV_LIB  DEV_Config.c -o DEV_Config.o
gcc -c -Wall -g -D USE_DEV_LIB  dev_hardware_i2c.c -o dev_hardware_i2c.o
gcc -c -Wall -g -D USE_DEV_LIB  sysfs_gpio.c -o sysfs_gpio.o
gcc -c -Wall -g -D USE_DEV_LIB  PCA9685.c -o PCA9685.o
gcc -c -Wall -g -D USE_DEV_LIB  MotorDriver.c -o MotorDriver.o
gcc main.o DEV_Config.o dev_hardware_i2c.o sysfs_gpio.o PCA9685.o MotorDriver.o  -o assignment3 -lpigpio -lrt -lpthread -lm
pi40@raspberrypi40:~/EmbeddingLinux/csc-615-assignment-3-startyourmotors-khatri5034 $ sudo ./assignment3
```

Kiran Khatri
Github: khatri5034

ID: 923750019
CSC615 Embedded Linux

Screen shot(s) of the execution of the program:

```
pi40@raspberrypi40:~/EmbeddingLinux/csc-615-assignment-3-startyourmotors-khatri5034 $ sudo ./assignment3
USE_DEV_LIB
Debug : prescaleval = 66.816840
Debug : prescaleval = 66.816840
Waiting for button to be pressed.
Forward 100% for 2 seconds
Gradually slowing to 15%
Stop for 1 second
Reverse gradually to 100%
Cleaning up and exiting
pi40@raspberrypi40:~/EmbeddingLinux/csc-615-assignment-3-startyourmotors-khatri5034 $
```